WEEK 08

JSON values and dictionary checks

Python foundations

JSON values and dictionary checks

JSON text, Python values and validation order

JSON decodes into values

JSON text can become a list, dictionary, number or other value.

```
import json
print(json.loads("[1, 2]"))  # [1, 2]
```

Shape before fields

Among JSON values, only a dictionary has named fields.

```
data = [1, 2]
print(isinstance(data, dict))  # False
```

Exact type before range

An exact-integer contract excludes Boolean values.

```
print(type(True) is int)  # False
print(type(4) is int)     # True
```

Reading a form is only the first check

1. Decode

json.loads(text)

→

2. Check shape

type(value) is dict

→

3. Check field

"count" + type + range

`[]`

Valid JSON

Wrong shape

`{"count": null}`

Field is present

Value is invalid

`{"count": 4}`

Correct shape

Valid count

```
type(count) is int and count > 0  # excludes True, "4", 0
```

Parse → shape → field → value. Each check answers a different question.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
json.loads(text)	Parse JSON text	Obtain Python values
isinstance(x, dict)	Check the shape	Before dictionary methods
mapping.get(key)	Read a field	Handle a missing key
type(x) is int	Check an exact integer	Exclude bool, then check range

Python Basics · sections 05, 09, 12, 13 · use only when you need a lookup.

Decode one JSON value at a time

```
01 import json
02 print(type(json.loads('{"count": 4}')).__name__)
03 print(type(json.loads('[]')).__name__)
04 print(type(json.loads('null')).__name__)
```

1. JSON text is a string; json.loads returns a Python value.

Check shape before reading a key

```
01 import json
02 candidate = json.loads('[]')
03 if type(candidate) is not dict:
04     print("Object required")
05 else:
06     print(candidate.get("count"))
```

1. Only the dictionary has named fields.

Distinguish a missing key from an invalid value

```
01 import json
02 candidate = json.loads('{"count": null}')
03 print("count" in candidate)
04 value = candidate.get("count")
05 print(value)
06 print(type(value) is int and value > 0)
```

1. The in operator checks whether a dictionary has a key.

Catch malformed JSON specifically

```
01 import json
02 text = "{"
03 try:
04     value = json.loads(text)
05 except json.JSONDecodeError:
06     print("Invalid JSON")
```

1. A JSONDecodeError means parsing could not produce a value.

JSON parsing does not guarantee a dictionary

```
01 import json
02 for text in ['{"count": 4}', '[4]', 'null']:
03     value = json.loads(text)
04     print(repr(value), type(value).__name__)
```

1. Expected types: dict, list, NoneType.
2. All three inputs are valid JSON.
3. Only the dictionary supports field lookup with .get().

JSON values and exact Python types

```
01 import json
02 candidate = json.loads('{"containers": true}')
03 count = candidate.get("containers")
04 valid = type(count) is int and count > 0
05 print(count, valid)
06 assert valid is False
```

1. json.loads() converts JSON values.
2. type(count) is int excludes bool.
3. Check the range after the type.

JSON true becomes bool, which is different from an exact int.

Dictionary lookup and a rejected transition

```
def next_state(state, event):
    allowed = {"reserved": "collected",
               "collected": "delivered"}
    if allowed.get(state) != event:
        raise ValueError("out of order or repeated")
    return event
state = "reserved"
try:
    state = next_state(state, "delivered")
except ValueError:
    pass  # Deliberate failed transition.
assert state == "reserved"
```

1. Use .get() to look up the allowed value.
2. Raise when the condition fails.
3. Check the value after except.

A raised exception prevents assignment of a new result.

Hands-on practice

Week_08.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

Reports and donor review queue

The project need

1

Need

The coordinator needs to distinguish promises from actual events.

2

Operation

Compare pickup records and review incoming messages before using their claims.

3

Result

The coordinator knows which pickups need follow-up.

Collected, but not delivered → follow up.

Use events to explain each lifecycle stage

1

Reserved

Accepted request
Availability reduced

2

Collected

Explicit collection event
For an accepted request

3

Delivered

Explicit delivery event
Collection must come first

These measures overlap. Do not add them as separate amounts.

Practice offline, then record required HTTP evidence

Offline simulation

No socket is opened. Tests exercise candidate validation and a deliberately raised failure.

Required classroom HTTP

Set the flag True. Send ID and original text; record actual responses and the six-message queue.

Simulation tests handling; only an actual request tests the HTTP path.

GROUP BY: aggregate related rows

```sql
SELECT batch_id, SUM(containers) AS containers,
       SUM(containers * ppc) AS planned_portions
FROM lines
GROUP BY batch_id
ORDER BY batch_id;
```

1. Miniature T10: three containers, nine portions
2. Miniature T20: one container, four portions
3. Group by request_id to reconcile another way

The grouping key determines what one output row means.

EXISTS: filter without multiplying rows

```sql
SELECT COALESCE(SUM(r.planned_portions), 0)
FROM reservations r
WHERE EXISTS (
  SELECT 1 FROM events e
  WHERE e.request_id = r.request_id
    AND e.event_type = 'collected'
);
```

1. The inner query tests for a match.
2. Each outer row contributes once.
3. COALESCE replaces a null total with zero.

EXISTS filters outer rows without multiplying them.

try / except: retain failure information

```
from urllib.error import HTTPError
queue = []
try:
    candidate = fetch(message_id)  # supplied context
except HTTPError as error:
    queue.append({"message_id": message_id,
                  "reason": f"HTTP {error.code}",
                  "next_action": "coordinator follow-up"})
```

1. fetch is supplied by the example context.
2. except binds the HTTPError to error.
3. append() adds a structured failure record.

Catch a specific failure and retain information needed by the caller.

Project task

Supplied starting point

Canonical CSV/database, six messages, offline fetch and local HTTP replay service are supplied.

You implement / do

Implement grouped reports and field/shape checks. Retain source, outcome, reasons and action for M1–M6; run offline and actual local HTTP evidence.

Check the evidence

Metrics: offered 94, available 34, reserved 60, collected 42, delivered 29. Reconcile 34+18+13+29=94; M6 HTTP503 needs follow-up; intake leaves DB unchanged.

Keep offline and actual HTTP evidence separate; pitch a small Week 9 change.

Project checks

Step	What to show
Implementation	Implement metrics and queue; record actual local HTTP.
Checks	94/34/60/42/29; M6 follow-up; intake leaves DB unchanged.

Keep offline and actual HTTP evidence separate; pitch a small Week 9 change.

Further reading in English

1. ## Python urllib.request documentation

   https://docs.python.org/3/library/urllib.request.html

2. ## Python json documentation

   https://docs.python.org/3/library/json.html

3. ## SQLite aggregate functions

   https://www.sqlite.org/lang_aggfunc.html

Read one relevant section, then bring an example and a question.